

INHA UNIVERSITY TASHKENT

FALL SEMESTER 2024

SOC 3010

OPERATING SYSTEM

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

SOC 3010

OPERATING SYSTEM

WEEK 10 LECTURE 2

INTER PROCESS COMMUNICATION (IPC)

WISH YOU ALL THE BEST

INTER PROCESS COMMUNICATION

- ★ **User Mode processes can synchronize their actions and exchange data**
- ★ **All Unix kernels include a set of system calls that supports process communication without interacting with the filesystem**
- ★ **Unix systems offers basic mechanisms to allow inter-process communication:**
 - **Pipes and FIFOs (named pipes)**
 - **Semaphores**
 - **Messages**
 - **Shared memory regions**
 - **Sockets**

INTER PROCESS COMMUNICATION

❑ Pipes and FIFOs (named pipes)

- ★ Best suited to implement producer/consumer interactions among processes. Some processes fill the pipe with data, while others extract data from the pipe.

❑ Semaphores

- ★ Represent, as the name implies, the User Mode version of the kernel semaphores

❑ Messages

- ★ Allow processes to exchange messages (short blocks of data) by reading and writing them in predefined message queues. The Linux kernel offers two different versions of messages: System V IPC messages and POSIX messages

INTER PROCESS COMMUNICATION

❑ Shared memory regions

- ★ Allow processes to exchange information via a shared block of memory. In applications that must share large amounts of data, this can be the most efficient form of process communication.

❑ Sockets

- ★ Allow processes on different computers to exchange data through a network. Sockets can also be used as a communication tool for processes located on the same host computer; the X Window System graphic interface, for instance, uses a socket to allow client programs to exchange data with the X server.

PIPES

- ★ Pipes are an interprocess communication mechanism that is provided in all flavors of Unix.
- ★ A pipe is a one-way flow of data between processes: all data written by a process to the pipe is routed by the kernel to another process, which can thus read it.
- ★ In Unix command shells, pipes can be created by means of the | operator.
- ★ For instance, the following statement instructs the shell to create two processes connected by a pipe:
\$ ls | more
- ★ The standard output of the first process, which executes the ls program, is redirected to the pipe; the second process, which executes the more program, reads its input from the pipe.

PIPES

- ★ Note that the same results can also be obtained by issuing two commands such as the following:

\$ ls > temp

\$ more < temp

- ★ The first command redirects the output of **ls** into a regular file; then the second command forces **more** to read its input from the same file.
- ★ Instead of using temporary files, pipe is usually more convenient due to the following reasons:
 - **The shell statement is much shorter and simpler.**
 - **There is no need to create temporary regular files, which must be deleted later.**

□ Using a Pipe

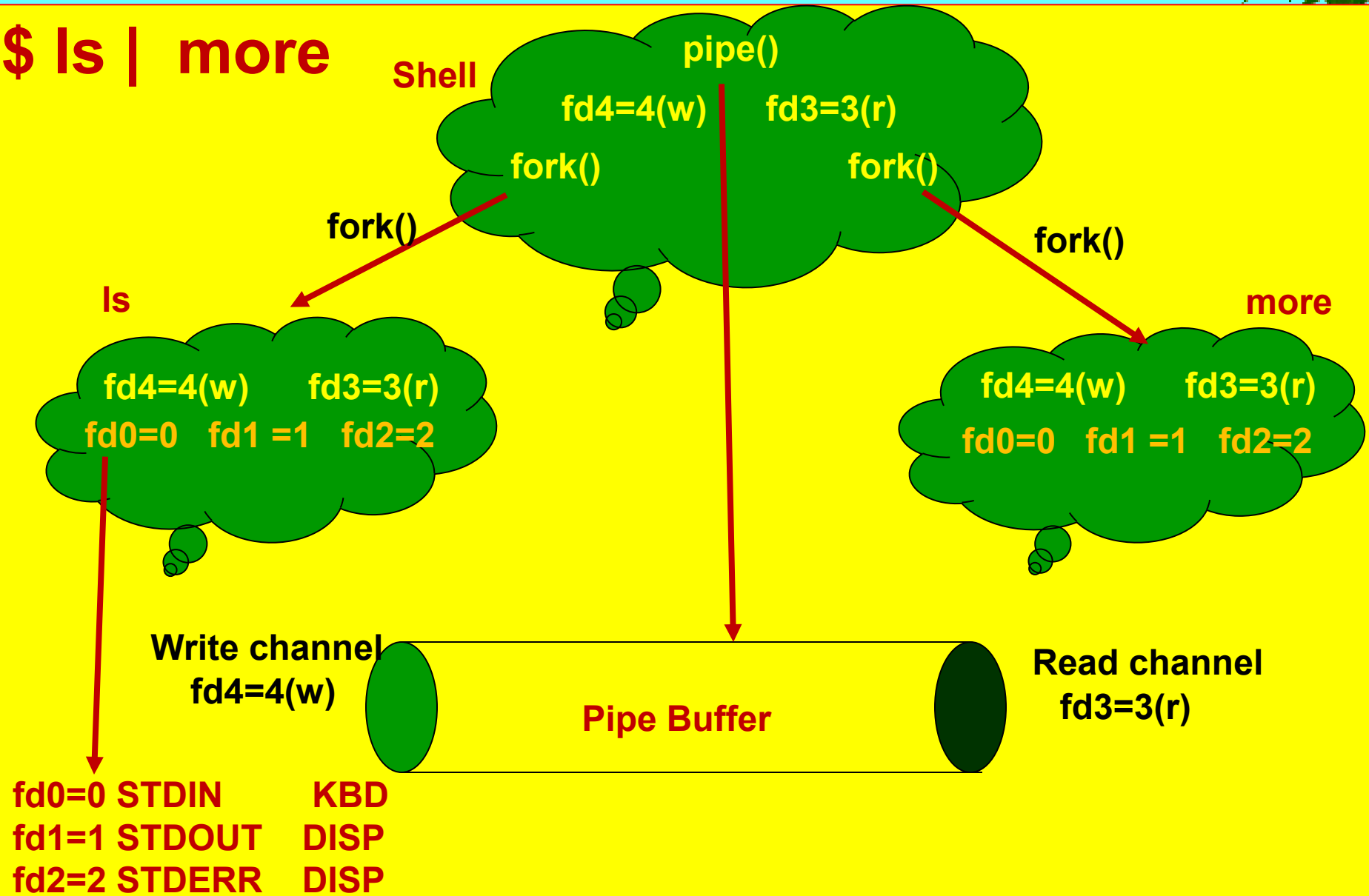
- * Pipes may be considered open files that have no corresponding image in the mounted filesystems.
- * A process creates a new pipe by means of the **pipe() system call**, which returns a pair of file descriptors; the process may then pass these descriptors to its descendants through **fork()**, thus sharing the pipe with them.
- * The processes can read from the pipe by using the **read() system call** with the first file descriptor; likewise, they can write into the pipe by using the **write() system call** with the second file descriptor.
- * **POSIX defines only half-duplex pipes**, so even though the **pipe() system call** returns two file descriptors, each process must close one before using the other.
- * **If a two-way flow of data is required, the processes must use two different pipes by invoking pipe() twice.**

□ Using a Pipe

- ★ Several Unix systems, such as System V Release 4, implement full-duplex pipes.
- ★ In a full-duplex pipe, both descriptors can be written into and read from, thus there are two bidirectional channels of information.
- ★ Linux adopts yet another approach: each pipe's file descriptors are still one-way, but it is not necessary to close one of them before using the other.

PIPES

\$ ls | more



PIPES

\$ ls | more

Shell

pipe()

fd4=4(w)

fd3=3(r)

fork()

close(fd4))

fork()

close(fd3)

fork()

fork()

ls

more

fd4=4(w) fd3=3(r)

dup2(4,1)

fd0=0 fd1=4 fd2=2

close(fd4)) close(fd3)

execve("ls", argv)

fd4=4(w) fd3=3(r)

dup2(3,0)

fd0=3 fd1=1 fd2=2

close(fd4)) close(fd3)

execve("more", argv)

Write channel

fd4=4(w)

Read channel

fd3=3(r)

Pipe Buffer

fd0=0 STDIN
fd1=1 STDOUT
fd2=2 STDERR

PIPES

- * When the command shell interprets the `ls | more` statement, it essentially performs the following actions:
 1. Invokes the `pipe( )` system call; let's assume that `pipe( )` returns the file descriptors 3 (the pipe's read channel) and 4 (the write channel).
 2. Invokes the `fork( )` system call twice.
 3. Invokes the `close( )` system call twice to release file descriptors 3 and 4.
- * The first child process, which must execute the `ls` program, performs the following operations:
 1. Invokes `dup2(4,1)` to copy file descriptor 4 to file descriptor 1. From now on, file descriptor 1 refers to the pipe's write channel.
 2. Invokes the `close( )` system call twice to release file descriptors 3 and 4.
 3. invokes the `execve( )` system call to execute the `ls` program The program writes its output to the file that has file descriptor 1 (the standard output); i.e., it writes into the pipe.

PIPES

- * **The second child process must execute the more program; therefore, it performs the following operations:**
 1. **Invokes `dup2(3,0)` to copy file descriptor 3 to file descriptor 0. From now on, file descriptor 0 refers to the pipe's read channel.**
 2. **Invokes the `close( )` system call twice to release file descriptors 3 and 4.**
 3. **Invokes the `execve( )` system call to execute `more`. By default, that program reads its input from the file that has file descriptor 0 (the standard input); i.e., it reads from the pipe.**

PIPES

- ★ Many Unix systems provide, besides the `pipe( )` system call, two wrapper functions named **`popen( )`** and **`pclose( )`** that handle all the dirty work usually done when using pipes.
- ★ Once a pipe has been created by means of the **`popen( )`** function, it can be used with the high-level I/O functions included in the C library (`fprintf( )`, `fscanf( )`, and so on).
- ★ In Linux, **`popen( )`** and **`pclose( )`** are included in the C library.
- ★ The `popen( )` function receives two parameters: the filename pathname of an executable file and a type string specifying the direction of the data transfer.
- ★ It returns the pointer to a FILE data structure.

- * **The popen() function essentially performs the following operations:**
 - 1. Creates a new pipe by using the pipe() system call.**
 - 2. Forks a new process, which in turn executes the following operations:**
 - a. If type is r, it duplicates the file descriptor associated with the pipe's write channel as file descriptor 1 (standard output); otherwise, if type is w, it duplicates the file descriptor associated with the pipe's read channel as file descriptor 0 (standard input).**
 - b. Closes the file descriptors returned by pipe().**
 - c. Invokes the execve() system call to execute the program specified by filename.**
 - 3. If type is r, it closes the file descriptor associated with the pipe's write channel; otherwise, if type is w, it closes the file descriptor associated with the pipe's read channel.**
 - 4. Returns the address of the FILE file pointer that refers to whichever file descriptor for the pipe is still open.**

- ★ After the **popen() invocation**, parent and child can exchange information through the pipe: the parent can read (if type is r) or write (if type is w) data by using the FILE pointer returned by the function.
- ★ The data is written to the standard output or read from the standard input, respectively, by the program executed by the child process.
- ★ The **pclose() function** (which receives the file pointer returned by popen() as its parameter) simply invokes the wait4() system call and waits for the termination of the process created by popen().

□ Pipe Data Structures

- ★ Once a pipe is created, a process uses the **read()** and **write()** VFS system calls to access it.
- ★ Therefore, for each pipe, the kernel creates an **inode object plus two file objects one for reading and the other for writing.**
- ★ When a process wants to read from or write to the pipe, it must use the proper file descriptor.
- ★ When the inode object refers to a pipe, its **i_pipe** field points to a **pipe_inode_info** structure shown in Table

PIPE

The pipe_inode_info structure

Type	Field	Description
struct wait_queue *	wait	Pipe/FIFO wait queue
unsigned int	nrbufs	Number of buffers containing data to be read
unsigned int	curbuf	Index of first buffer containing data to be read
struct pipe_buffer [16]	bufs	Array of pipe's buffer descriptors
struct page *	tmp_page	Pointer to a cached page frame
unsigned int	start	Read position in current pipe buffer
unsigned int	readers	Flag for (or number of) reading processes
unsigned int	writers	Flag for (or number of) writing processes
unsigned int	waiting_writers	Number of writing processes sleeping in the wait queue
unsigned int	r_counter	Like readers, but used when waiting for a process that reads from the FIFO
unsigned int	w_counter	Like writers, but used when waiting for a process that writes into the FIFO
struct fasync_struct *	fasync_readers	Used for asynchronous I/O notification via signals
struct fasync_struct *	fasync_writers	Used for asynchronous I/O notification via signals

PIPES

- ★ Besides one inode and two file objects, each pipe has its own set of pipe buffers .
- ★ Essentially, a pipe buffer is a page frame that contains data written into the pipe and yet to be read.
- ★ Each pipe makes use of 16 pipe buffers which greatly enhances the performance of User Mode applications that write large chunks of data in a pipe.
- ★ The bufs field of the pipe_inode_info data structure stores an array of 16 pipe_buffer objects, each of which describes a pipe buffer. The fields of this object are shown in Table

PIPE

Field of pipe_buffer object

Type	Field	Description
struct page *	page	Address of the descriptor of the page frame for the pipe buffer
unsigned int	offset	Current position of the significant data inside the page frame
unsigned int	len	Length of the significant data in the pipe buffer
struct pipe_buf_operations *	ops	Address of a table of methods relative to the pipe buffer (NULL if the pipe buffer is empty)

PIPES

- ★ **The 16 pipe buffers can be seen as a global, circular buffer: writing processes keep adding data to this large buffer, while reading process keep removing them.**
- ★ **The number of bytes currently written in all pipe buffers and yet to be read is the so-called pipe size.**
- ★ **For reasons of efficiency, the data yet to be read can be spread among several partially filled pipe buffers: in fact, each write operation may copy the data in a fresh, empty pipe buffer if the previous pipe buffer has not enough free space to store the new data.**

Kernel must keep track of:

1. The **pipe buffer** that includes the next byte to be read, and the corresponding offset inside the page frame.

The index of this pipe buffer is stored in the `curbuf` field of the `pipe_inode_info` data structure, while the offset is stored in the `offset` field of the corresponding `pipe_buffer` object.

2. The **first empty pipe buffer**. Its index can be computed by adding (modulo 16) the index of the current pipe buffer, which is stored in the `curbuf` field of the `pipe_inode_info` data structure, and the number of pipe buffers with significant data, which is stored in the `nrbufs` field.

To avoid race conditions on the pipe's data structures, the kernel makes use of the **`i_sem semaphore`** included in the inode object.

pipefs special filesystem

- * A pipe is implemented as a set of VFS objects, which have no corresponding disk images.
- * In Linux, these VFS objects are organized into the **pipefs** special filesystem to expedite their handling
- * Because this filesystem has no mount point in the system directory tree, users never see it.
- * Pipes are fully integrated in the VFS layer, and the kernel can handle them in the same way as named pipes or FIFOs, which truly exist as files recognizable to end users
- * The `init_pipe_fs( )` function, typically executed during kernel initialization, registers the pipefs filesystem and mounts it

```
struct file_system_type pipe_fs_type;  
pipe_fs_type.name = "pipefs";  
pipe_fs_type.get_sb = pipefs_get_sb;  
pipe_fs.kill_sb = kill_anon_super;  
register_filesystem(&pipe_fs_type);  
pipe_mnt = do_kern_mount("pipefs", 0, "pipefs", NULL);
```

- * The mounted filesystem object that represents the root directory of pipefs is stored in the `pipe_mnt` variable.

Creating and Destroying a Pipe

❑ Creating a Pipe

- * The **pipe()** system call is serviced by the **sys_pipe()** function, which in turn invokes the **do_pipe()** function.
- * To create a new pipe, **do_pipe()** performs the following operations:
 1. Invokes the **get_pipe_inode()** function, which allocates and initializes an inode object for the pipe in the pipefs filesystem.
In particular, this function executes the following actions:
 - a. Allocates a new inode in the pipefs filesystem.
 - b. Allocates a **pipe_inode_info** data structure and stores its address in the **i_pipe** field of the inode.
 - c. Sets the **curbuf** and **nrbufs** fields of the **pipe_inode_info** structure to 0; also, fills with zeros all fields of the pipe buffer objects in the **bufs** array.
 - d. Initializes the **r_counter** and **w_counter** fields of the **pipe_inode_info** structure to 1.
 - e. Sets the **readers** and **writers** fields of the **pipe_inode_info** structure to 1

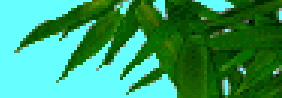
Creating a Pipe

- 2. Allocates a file object and a file descriptor for the read channel of the pipe, sets the f_flag field of the file object to O_RDONLY, and initializes the f_op field with the address of the read_pipe_fops table.**
- 3. Allocates a file object and a file descriptor for the write channel of the pipe, sets the flag field of the file object to O_WRONLY, and initializes the f_op field with the address of the write_pipe_fops table.**
- 4. Allocates a dentry object and uses it to link the two file objects and the inode object then inserts the new inode in the pipefs special filesystem.**
- 5. Returns the two file descriptors to the User Mode process**

PROCESSES

- ★ The process that issues a pipe() system call is initially the only process that can access the new pipe, both for reading and writing.
- ★ To represent that the pipe has both a reader and a writer, the readers and writers fields of the pipe_inode_info data structure are initialized to 1.
- ★ In general, each of these two fields is set to 1 only if the corresponding pipe's file object is still opened by a process; the field is set to 0 if the corresponding file object has been released, because it is no longer accessed by any process.
- ★ Forking a new process does not increase the value of the readers and writers fields, so they never rise above 1; however, it does increase the value of the usage counters of all file objects still used by the parent process
- ★ Thus, the objects are not released even when the parent dies, and the pipe stays open for use by the children.

Destroying a Pipe



- ★ Whenever a process invokes the **close()** system call on a file descriptor associated with a pipe, the kernel executes the **fput()** function on the corresponding file object, which decreases the usage counter.
- ★ If the counter becomes 0, the function invokes the release method of the file operations
- ★ Depending on whether the file is associated with the read or write channel, the release method is implemented by either **pipe_read_release()** or **pipe_write_release()**; both functions invoke **pipe_release()**, which sets either the readers field or the writers field of the **pipe_inode_info** structure to 0.
- ★ The function checks whether both the readers and writers fields are equal to 0; in this case, it invokes the pipe buffer's release method of all pipe buffers, thus releasing to the buddy system all pipe's page frames; moreover, the function releases the cached page frame pointed to by the **tmp_page** field.
- ★ Otherwise, if either the readers field or the writers field is not zero, the function wakes up the processes sleeping in the pipe's wait queue so they can recognize the change in the pipe state.



Reading from a Pipe

- * A process wishing to get data from a pipe issues a **read()** system call, specifying the file descriptor associated with the pipe's reading end.
- * The kernel ends up invoking the read method found in the file operation table associated with the proper file object. In the case of a pipe, the entry for the read method in the read_pipe_fops table points to the **pipe_read()** function.
- * The **pipe_read()** function is quite involved, because the POSIX standard specifies several requirements for the pipe's read operations.
- * The system call might block the current process in two cases:
 - The pipe buffer is empty when the system call starts.
 - The pipe buffer does not include all requested bytes, and a writing process was previously put to sleep while waiting for space in the buffer.
- * Notice that the read operation can be nonblocking: in this case, it completes as soon as all available bytes (even none) are copied into the user address space.

READING FROM A PIPE

- ★ Table summarizes the expected behavior of a `read( )` system call that requests `n` bytes from a pipe that has a pipe size (number of bytes in the pipe buffers yet to be read) equal to `p`.

Pipe Size p	At least one writing process			No writing process
	Blocking read		Nonblocking read	
	Sleeping writer	No Sleeping writer		
p=0	Copy n bytes and return n, Waiting for data when the pipe buffer is empty	Wait for some data, copy it, and return its size	Return -EAGAIN	Return 0
0 < p < n		Copy p bytes and return p : 0 bytes are left in the pipe buffer		
p >= n	Copy n bytes and return n : p-n bytes are left in the pipe buffer			

Reading n bytes from a pipe

Writing into a Pipe

❑ Writing into a Pipe

- ★ A process wishing to put data into a pipe issues a **write()** system call, specifying the file descriptor for the writing end of the pipe.
- ★ The kernel satisfies this request by invoking the write method of the proper file object; the corresponding entry in the **write_pipe_fops** table points to the **pipe_write()** function.
- ★ In particular, the standard requires that write operations involving a small number of bytes must be atomically executed.
- ★ More precisely, if two or more processes are concurrently writing into a pipe, each write operation involving fewer than 4,096 bytes (the pipe buffer size) must finish without being interleaved with write operations of other processes to the same pipe.
- ★ However, write operations involving more than 4,096 bytes may be nonatomic and may also force the calling process to sleep.

Writing into a Pipe

- ★ Table summarizes the behavior, specified by the POSIX standard, of a **write()** system call that requested to write **n** bytes into a pipe having **u** unused bytes in its buffer.

Available buffer space u	At least one reading process		No reading process
	Blocking write	Nonblocking write	
u < n <= 4096	Wait until n-u bytes are freed, copy n bytes, and return n.	Return - EAGAIN	Send SIGPIPE and Return - EPIPE
n > 4096		If u > 0, Copy u bytes and return u, return-EAGAIN	
u >= n	Copy n bytes and return n		

Writing **n** bytes to a pipe

FIFOs

- ★ Although pipes are a simple, flexible, and efficient communication mechanism, they have one main drawback namely, that there is no way to open an already existing pipe.
- ★ This makes it impossible for two arbitrary processes to share the same pipe, unless the pipe was created by a common ancestor process.
- ★ This drawback is substantial for many application programs.
- ★ Consider, for instance, a database engine server, which continuously polls client processes wishing to issue some queries and which sends the results of the database lookups back to them.
- ★ Each interaction between the server and a given client might be handled by a pipe.
- ★ However, client processes are usually created on demand by a command shell when a user explicitly queries the database; **server and client processes thus cannot easily share a pipe.**

FIFOs

- ★ To address such limitations, Unix systems introduce a special file type called a named pipe or FIFO (which stands for "first in, first out;" the first byte written into the special file is also the first byte that is read).
- ★ Each FIFO is much like a pipe: rather than owning disk blocks in the filesystems, an opened FIFO is associated with a kernel buffer that temporarily stores the data exchanged by two or more processes.
- ★ A FIFO can be accessed by every process, because the FIFO filename is included in the system's directory tree.
- ★ Thus, in our example, the communication between server and clients may be easily established by using FIFOs instead of pipes.
- ★ The server creates, at startup, a FIFO used by client programs to make their requests.
- ★ Each client program creates, before establishing the connection, another FIFO to which the server program can write the answer to the query and includes the FIFO's name in the initial request to the server.

FIFOs

- ★ In Linux, FIFOs and pipes are almost identical and use the same `pipe_inode_info` structures.
- ★ As a matter of fact, the read and write file operation methods of a FIFO are implemented by the same `pipe_read( )` and `pipe_write( )` functions
- ★ Actually, there are only two significant differences:
 - FIFO inodes appear on the system directory tree rather than on the `pipefs` special filesystem.
 - FIFOs are a bidirectional communication channel; that is, it is possible to open a FIFO in read/write mode.

Creating and Opening a FIFO

❑ Creating and Opening a FIFO

- ★ A process creates a FIFO by issuing a **mknod()** system call passing to it as parameters the pathname of the new FIFO and the value **S_IFIFO** (0x10000) logically ORed with the permission bit mask of the new file.
- ★ POSIX introduces a function named **mkfifo()** specifically to create a FIFO.
- ★ This call is implemented in Linux, as in System V Release 4, as a C library function that invokes **mknod()**.
- ★ Once created, a FIFO can be accessed through the usual **open()**, **read()**, **write()**, and **close()** system calls, but the VFS handles it in a special way, because the FIFO inode and file operations are customized and do not depend on the filesystems in which the FIFO is stored.

FIFOs

- ★ When a process opens a FIFO, the VFS performs the same operations as it does for device files
- ★ The inode object associated with the opened FIFO is initialized by a filesystem-dependent **read_inode superblock method**; this method always checks whether the inode on disk represents a special file, and invokes, if necessary, the **init_special_inode()** function.
- ★ In turn, this function sets the **i_fop** field of the inode object to the **address of the def_fifo_fops table**.
- ★ Later, the kernel sets the file operation table of the file object to **def_fifo_fops**, and executes its open method, which is implemented by **fifo_open()**.

FIFOs

- * The **fifo_open()** function initializes the data structures specific to the FIFO; in particular, it performs the following operations:
 1. Acquires the **i_sem inode** semaphore.
 2. Checks the **i_pipe field** of the inode object; if it is NULL, it allocates and it initializes a new **pipe_inode_info structure**
 3. Depending on the access mode specified as the parameter of the **open() system call**, it initializes the **f_op field** of the file object with the address of the proper file operation table

FIFO's file operations

<u>Access type</u>	<u>File operations</u>	<u>Read method</u>	<u>Write method</u>
Read-only	read_fifo_fops	pipe_read()	bad_pipe_w()
Write-only	write_fifo_fops	bad_pipe_r()	pipe_write()
Read/write	rdwr_fifo_fops	pipe_read()	pipe_write()

FIFOs

❑ Access Mode – Read only or Read/Write

- ★ If the access mode is either read-only or read/write, it adds one to the readers and r_counter fields of the pipe_inode_info structure.
- ★ Moreover, if the access mode is read-only and there is no other reading process, it wakes up any writing process sleeping in the wait queue

❑ Access Mode – Write only or Read/Write

- ★ If the access mode is either write-only or read/write, it adds one to the writers and w_counter fields of the pipe_inode_info structure.
- ★ Moreover, if the access mode is write-only and there is no other writing process, it wakes up any reading process sleeping in the wait queue.
- ★ If there are no readers or no writers, it decides whether the function should block or terminate returning an error code

❑ Behavior of the `fifo_open( )` function

<u>Access type</u>	<u>Blocking</u>	<u>Nonblocking</u>
Read-only, with writers	Successfully return	Successfully return
Read-only, no writer	Wait for a writer	Successfully return
Write-only, with readers	Successfully return	Successfully return
Write-only, no reader	Wait for a reader	Return -ENXIO
Read/write	Successfully return	Successfully return

- * Releases the inode semaphore, and terminates, returning 0 (success).
- * The FIFO's three specialized file operation tables differ mainly in the implementation of the read and write methods.
- * If the access type allows read operations, the read method is implemented by the `pipe_read( )` function. Otherwise, it is implemented by `bad_pipe_r( )`, which only returns an error code.
- * Similarly, if the access type allows write operations, the write method is implemented by the `pipe_write( )` function; otherwise, it is implemented by `bad_pipe_w( )`, which also returns an error code.

INTERRUPTS & EXCEPTIONS

□ Questions

- * Explain what is meant by Inter Process Communication (IPC).
- * State and Explain the six basic mechanisms offered by linux systems to allow Inter Process Communication.
- * Explain the following :
 - a) Pipes
 - b) FIFOs
 - c) Semaphores
 - d) Message Queues
 - e) Shared Memory
 - f) Sockets
- * Explain what is meant by a pipe.
- * Explain the following functions :
 - a) pipe() b) execve() c) dup2() d) popen() e) pclose() f) pipe_read() g) pipe_write()
- * Describe all the actions essentially performed by the command shell when the command statement `ls | more` is interpreted and executed after invoking the pipe command. Also clearly indicate the actions performed by the two child processes created.
- * Describe the operations performed by the function `popen()`.
- * One pipe buffer consists of page frame(s)
 - a) 1 b) 4 c) 8 d) 16
- * Each pipe makes use of pipe buffers which greatly enhances the performance of User Mode applications that write large chunks of data in a pipe.
 - a) 1 b) 4 c) 8 d) 16
- * Describe pipefs special filesystem. Indicate where this is located.
- * State the three limitations of a pipe.
- * Explain what is meant by named pipe or FIFO.
- * Distinguish between Pipes & FIFOs.
- * What are the advantages of FIFO over unnamed pipe?
- * Explain the operations performed by the following functions :
 - a) `mknod()` b) `mkfifo()`

SOC 3010 OPERATING SYSTEMS

Reading Assignments

Chapter 19 of Text Book

Text Book

➤ Understanding the Linux Kernel

- Daniel P/ Bovet and Marco Cesati
Edition, O' Reilly Media 2005, ISBN 978-0596005658**

SOC 3010

OPERATING SYSTEM

END OF
WEEK 10 LECTURE 2

INTER PROCESS
COMMUNICATION

WISH YOU ALL THE BEST

INHA UNIVERSITY TASHKENT

FALL SEMESTER 2024

SOC 3010

OPERATING SYSTEM

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

